

Uso de solid en el ejercicio 3

Principio S

Para este principio nos aseguramos de que cada clase tuviera una sola tarea bien definida. Como la clase `TextNormalizer` que solo se encarga de limpiar el texto quitando puntos, comas, números y acentos mediante el método `normalize()`. Y como la clase `HashTable` que se dedica únicamente a guardar las palabras, evitar que se repitan usando buckets y llevar el conteo de frecuencias mediante `add_or_increment()`.

Principio O

Aquí aplicamos este principio creando una clase base abstracta `SorterStrategy` con el método abstracto `sort()`. Gracias a esta clase base tenemos como subclases `MergeSortStrategy` y `QuickSortStrategy`. Como el código está abierto a extenderse pero cerrado a modificaciones, cuando necesitemos agregar otro algoritmo solo tendremos que crear una nueva subclase que herede de `SorterStrategy` y así no tendremos que modificar la lógica general del programa.

Principio L

Este principio se cumple pues al ser `MergeSortStrategy` y `QuickSortStrategy` subclases de `SorterStrategy` van a hacer exactamente lo que la clase base define, es decir, recibir una lista de palabras y regresarla ordenada alfabéticamente. Lo que nos puede permitir intercambiar `MergeSortStrategy` por `QuickSortStrategy` en cualquier momento de la ejecución para tomar los tiempos sin que el programa falle o altere el resultado.

Principio I

Para este principio nos encargamos de dividir las responsabilidades en partes pequeñas, en lugar de crear una clase gigante que haga todo. Por ejemplo, el módulo que mide los tiempos de ejecución solo interactúa con la interfaz de ordenamiento (`SorterStrategy`) para tomar las lecturas de forma rápida, pues no necesita saber cómo funciona la limpieza del texto en `TextNormalizer` ni cómo la tabla hash (`HashTable`) cuenta las palabras y elimina los duplicados.

Principio D

Este principio se utilizó en la estructura principal que coordina el diccionario, pues evitamos que dependiera directamente de los módulos de bajo nivel como lo son los algoritmos concretos. En lugar de instanciar `QuickSortStrategy()` dentro de la clase, esta recibe el algoritmo de ordenamiento desde afuera como una abstracción (`SorterStrategy`). Gracias a que le podemos "pasar" el algoritmo que queramos al momento de usarla, se facilita probar tanto `MergeSortStrategy` como `QuickSortStrategy` para tomar los tiempos sin tener que modificar ni una sola línea del código principal.